

Optymalizacja Kodu na Różne Architektury

Zadanie 1 - Normalizacja Tekstu

Cel zadania

Celem tego zadania jest implementacja funkcji, która przetwarza duże ciągi tekstowe (np. zawartość dużego pliku) wykonując szereg operacji normalizacyjnych oraz optymalizacyjnych.

Funkcja powinna:

- Usuwać znaki spoza zakresu drukowalnych (np. wszystkie znaki o kodzie poniżej 32 lub powyżej 126).
- Zamieniać sekwencje białych znaków (spacje, tabulatory, nowe linie) na pojedynczą spację.
- Konwertować wszystkie litery do małych liter.
- Konwertować interpunkcję na przecinki.
- Eliminować duplikaty wyrazów występujących jeden po drugim (np. "hello hello world" → "hello world").

Przykład

```
1 Source:      "Hello hello world... World."
2 Normalized:  "hello  world,,, ,"
3
4 Source:      "\t\n First.first, ,, second  second; third\n\n"
5 Normalized:  " first,,, ,, second , third"
6
7 Source:      "    leading space and trailing space    "
8 Normalized:  " leading space and trailing space "
9
10 Source:      "word1 word1, word2 word2"
11 Normalized:  "word1 , word2 "
12
13 Source:      "Hello\x07 World\x7F!"
14 Normalized:  "hello world,"
```

Dane techniczne

System Operacyjny

```
1 $ hostnamectl
2 Virtualization: wsl
3 Operating System: Ubuntu 24.04.2 LTS
4 Kernel: Linux 5.15.167.4-microsoft-standard-WSL2
```

Kompilator

```
1 $ g++ --version
2 g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
```

Procesor

```
1 $ lscpu
2 Architecture:          x86_64
3   CPU op-mode(s):      32-bit, 64-bit
4   Address sizes:        48 bits physical, 48 bits virtual
5   Byte Order:           Little Endian
6   CPU(s):               24
7   On-line CPU(s) list:  0-23
8   Vendor ID:            AuthenticAMD
9   Model name:           AMD Ryzen 9 7900X 12-Core Processor
10  CPU family:           25
11  Model:                 97
12  Thread(s) per core:   2
13  Core(s) per socket:   12
14  Socket(s):             1
15  Stepping:              2
16  BogoMIPS:              9399.78
17  Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36
18                        clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp
19                        lm constant_tsc rep_good nopl tsc_reliable nonsto p_tsc cpuid extd_apicid
20                        pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c
21                        rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse
22                        3dnowprefetch osvw topoext perfctr_core ssbd ibrs ibpb stibp vmmcall
23                        fsgsbase bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap
24                        avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveo pt
25                        xsavec xgetbv1 xsaves avx512_bf16 clzero xsaveerptr arat npt nrip_save
26                        tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold
27                        v_vmsave_vmload avx512vbmi umip avx512_vbmi2 gf ni vaes vpclmulqdq
28                        avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid fsrm
29  Caches (sum of all):
30    L1d:                  384 KiB (12 instances)
31    L1i:                  384 KiB (12 instances)
32    L2:                   12 MiB (12 instances)
33    L3:                   32 MiB (1 instance)
```

Obliczanie czasu oraz profilowanie

Czas został zmierzony dla zadanej liczby powtórzeń przy pomocy funkcji

```
1  const auto start = std::chrono::steady_clock::now();
2  for (int i = 0; i < repeats; ++i) {
3      normalized_content = normalize_text(content);
4  }
5  const auto finish = std::chrono::steady_clock::now();
6  const std::chrono::duration<double> elapsed_seconds = finish - start;
```

Fragment danych wejściowych (w zadaniu użyłem wersji o rozmiarach 100KB: 1000 powtórzeń, 100MB: 1 powtórzenie)

```
1  fpCEAO
2  fpCEAOGDF1 XBFFP } CueBMKk CueBMKk vRatVsQl
3  biNpsXGYw ITr BqHiYKNDq
4
5  <?>KQSVANXP <?>WnDAMnLjs hyoBoKZ xosRuQ vNfnGT : <?>: sGZHP uuDnkhNiBy
6  VSyL IPegMxiL MxiL lbhYaRAt lbhYaRAt CUVWEIFBDQ
7  - FgFVuKt <?>-
```

Do profilowania użyłem narzędzia perf.

Wersja bazowa

Implementacja polega na jednokrotnym przejściu przez tekst, mając zmienną na wynikowy tekst oraz zmienną na aktualnie ostatni i przetwarzany wyraz. Iteracja odbywa się litera po literze, dodając ją do odpowiedniej zmiennej w zależności od stanu.

```
1  /**
2   * @brief Normalizes a text string according to specified rules.
3   *
4   * Rules:
5   * 1. Remove non-printable ASCII characters (codes < 32 or > 126).
6   * 2. Replace sequences of whitespace characters with a single space.
7   * 3. Convert all letters to lowercase.
8   * 4. Convert punctuation to commas.
9   * 5. Eliminate consecutive duplicate words ("hi hi world" -> "hi world").
10  *
11  * @param text The input string to normalize.
12  * @return The normalized string.
13  */
14  std::string normalize_text(const std::string& text) {
15      std::string result;
16      std::string word, last_word;
17      enum State { NORMAL, COMMA, SPACE, DUPLICATE_SPACE } state = NORMAL;
18
19      for (char c : text) {
20          if (c < 32 || c > 126) continue; // rule 1
21
22          if (c == ' ') {
23              if (state == SPACE || state == DUPLICATE_SPACE) { // rule 2
24                  state = DUPLICATE_SPACE;
25              } else {
```

```

26     state = SPACE;
27 }
28 } else if (isalpha(c)) { // rule 3
29     c = tolower(c);
30     state = NORMAL;
31 } else if (ispunct(c)) { // rule 4
32     c = ',';
33     state = COMMA;
34 } else {
35     state = NORMAL;
36 }
37
38 if (state == COMMA || state == SPACE) {
39     // end of a word
40     if (!word.empty()) { // rule 5
41         if (word != last_word) result += word;
42         last_word = word;
43         word.clear();
44     }
45     result += c; // only delimiter
46 } else if (state == NORMAL) {
47     // in a word
48     word += c; // only alphanumeric
49 }
50 }
51 if (word != last_word) result += word; // last word
52
53 return result;
54 }

```

Czas działania dla plików o rozmiarze 100KB

```

1 normalize1 -00 [Benchmark (1000 iterations): 1.97064 seconds]
2 normalize1 -02 [Benchmark (1000 iterations): 0.64529 seconds]

```

Czas działania dla plików o rozmiarze 100MB

```

1 normalize1 -00 [100MB] [Benchmark (1 iterations): 1.98192 seconds]
2 normalize1 -02 [100MB] [Benchmark (1 iterations): 0.710283 seconds]

```

Wynik profilowania

1	54.56%	a.out	libc.so.6	[.] __mcount_internal
2	23.67%	a.out	libc.so.6	[.] _mcount
3	4.25%	a.out	a.out	[.] normalize_text(std::__cxx11::basic_string<char, ...
4	4.13%	a.out	a.out	[.] bool __gnu_cxx::operator!=<char const*, ...
5	2.57%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
6	2.12%	a.out	a.out	[.] __gnu_cxx::__normal_iterator<char const*, ...
7	2.09%	a.out	a.out	[.] __gnu_cxx::__normal_iterator<char const*, ...
8	1.51%	a.out	a.out	[.] __gnu_cxx::__normal_iterator<char const*, ...
9	1.11%	a.out	a.out	[.] std::__cxx11::basic_string<char, ...
10	1.10%	a.out	libc.so.6	[.] tolower
11	0.61%	a.out	a.out	[.] bool std::operator==<char, std::char_traits<char>, ...
12	0.50%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
13	0.29%	a.out	a.out	[.] bool std::operator!=<char, std::char_traits<char>, ...

Z analizy wyników możemy zaobserwować, że czas wykonania funkcji wygląda przyzwoicie, jedną z rzeczy, którą możemy spróbować zredukować, która nie jest wymagana w algorytmie, jest dużo operacji na iteratorze.

Wynik ten jednak widzimy, że jest znacznie poprawiony przy ustawieniu optymalizacji -O2, co może sugerować, że kompilator mógł zoptymalizować operacje na iteratorze.

Kolejną obserwacją jest liniowa złożoność czasowa, co jest zgodne z oczekiwaniami, ponieważ przechodzimy przez tekst tylko raz.

Co do zużycia pamięci, jest ona akceptowalna, ponieważ trzymamy tylko 2 zmienne pomocnicze ograniczone do rozmiaru wyrazu oraz wynikowy tekst, który jest potrzebny, ponieważ pozwala nam nie niszczyć oryginalnego tekstu oraz pozwala na efektywną obsługę powtarzających się wyrazów, co było by problematyczne przy usuwaniu ich w miejscu wymagającym przesunięcia wszystkich pozostałych wyrazów w takiej strukturze danych.

Zwyczajna pętla oraz prealokacja

W tej wersji zamieniono pętlę `range-for` na pętlę `for` oraz przetestowano prealokację zmiennych `result`, `word` oraz `last_word`.

```
1 result.reserve(text.length());
2 word.reserve(256);
3 last_word.reserve(256);
```

Użycie prealokacji zmiennych **nie przyniosło żadnej mierzalnej poprawy** w przypadku 1000 iteracji po pliku o rozmiarze 100KB oraz 1 iteracji po pliku o rozmiarze 100MB.

```
1 std::string normalize_text(const std::string& text) {
2     std::string result;
3     std::string word, last_word;
4     enum State { NORMAL, COMMA, SPACE, DUPLICATE_SPACE } state = NORMAL;
5
6     for (size_t i = 0; i < text.length(); ++i) {
7         char c = text[i];
8         if (c < 32 || c > 126) continue; // rule 1
9
10        if (c == ' ') {
11            if (state == SPACE || state == DUPLICATE_SPACE) { // rule 2
12                state = DUPLICATE_SPACE;
13            } else {
14                state = SPACE;
15            }
16        } else if (isalpha(c)) { // rule 3
17            c = tolower(c);
18            state = NORMAL;
19        } else if (ispunct(c)) { // rule 4
20            c = ',';
21            state = COMMA;
```

```

22     } else {
23         state = NORMAL;
24     }
25
26     if (state == COMMA || state == SPACE) {
27         // end of a word
28         if (!word.empty()) { // rule 5
29             if (word != last_word) result += word;
30             last_word = std::move(word);
31             word.clear();
32         }
33         result += c; // only delimiter
34     } else if (state == NORMAL) {
35         // in a word
36         word += c; // only alphanumeric
37     }
38 }
39 if (word != last_word) result += word; // last word
40
41 return result;
42 }

```

Czas działania dla plików o rozmiarze 100KB

```

1  normalize2 -O0 [Benchmark (1000 iterations): 1.31727 seconds]
2  normalize2 -O2 [Benchmark (1000 iterations): 0.635332 seconds]

```

Wynik profilowania

1	34.90%	a.out	a.out	[.] normalize_text(std::__cxx11::basic_string<char, ...
2	19.21%	a.out	libc.so.6	[.] __mcount_internal
3	9.18%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
4	7.85%	a.out	libc.so.6	[.] isalpha
5	4.57%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
6	4.54%	a.out	libc.so.6	[.] _mcount
7	4.33%	a.out	libc.so.6	[.] tolower
8	2.75%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
9	1.99%	a.out	a.out	[.] bool std::operator==<char, std::char_traits<char>, ...
10	1.51%	a.out	a.out	[.] bool std::operator!=<char, std::char_traits<char>, ...
11	1.15%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
12	1.12%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
13	0.96%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
14	0.66%	a.out	a.out	[.] std::__cxx11::basic_string<char, ...
15	0.58%	a.out	libc.so.6	[.] __memmove_avx512_unaligned_erms

W tym przypadku widzimy, że czas działania jest lepszy o około 30% w przypadku kompilacji bez optymalizacji. Czas z optymalizacją -O2 jest identyczny jak w przypadku poprzedniej wersji co potwierdza tezę, że kompilator mógł zoptymalizować operacje na iteratorze.

Dynamiczne zwiększanie rozmiaru zmiennych `result`, `word` oraz `last_word` nie jest problematyczne, ponieważ algorytm pracując na dużych plikach tekstowych stosunkowo szybko osiągnie wymagany rozmiar zmiennych.

Wersja bare-metal

W tej wersji użyto tablic znakowych w stylu C o stałym rozmiarze oraz ręcznych operacji na pamięci z użyciem `new` oraz `memcpy`. Program stał się znacznie mniej czytelny oraz wykorzystujemy niebezpieczne operację jak `strcmp`, co wymaga znacznie większej ostrożności. Kolejnym problemem jest założenie maksymalnej długości słowa.

```
1 char* normalize_text(const char* text, const ssize_t size) {
2     char* result = new char[size + 1]; // Null-terminated string
3     ssize_t result_len = 0;
4     char* word = new char[MAX_WORD_LEN];
5     ssize_t word_len = 0;
6     char* last_word = new char[MAX_WORD_LEN];
7     ssize_t last_word_len = 0;
8     enum State { NORMAL, COMMA, SPACE, DUPLICATE_SPACE } state = NORMAL;
9
10    for (size_t i = 0; i < size; ++i) {
11        char c = text[i];
12        if (c < 32 || c > 126) continue; // rule 1
13
14        if (c == ' ') {
15            if (state == SPACE || state == DUPLICATE_SPACE) { // rule 2
16                state = DUPLICATE_SPACE;
17            } else {
18                state = SPACE;
19            }
20        } else if (isalpha(c)) { // rule 3
21            c = tolower(c);
22            state = NORMAL;
23        } else if (ispunct(c)) { // rule 4
24            c = ',';
25            state = COMMA;
26        } else {
27            state = NORMAL;
28        }
29
30        if (state == COMMA || state == SPACE) {
31            // end of a word
32            if (word_len > 0) { // rule 5
33                if (strcmp(word, last_word) != 0) {
34                    memcpy(result + result_len, word, word_len + 1);
35                    result_len += word_len;
36                }
37                memcpy(last_word, word, word_len + 1);
38                last_word_len = word_len;
39                word_len = 0;
40            }
41            result[result_len++] = c; // only delimiter
42        } else if (state == NORMAL) {
43            // in a word
44            word[word_len++] = c; // only alphanumeric
45        }
46    }
47    if (strcmp(word, last_word) != 0) { // last word
48        memcpy(result + result_len, word, word_len + 1);
49        result_len += word_len;
50    }
```

```

51 // Clean up
52 result[result_len] = '\0';
53 delete[] word;
54 delete[] last_word;
55
56
57 return result;
58 }

```

Czas działania dla plików o rozmiarze 100KB

```

1 normalize3 -O0 [Benchmark (1000 iterations): 0.755573 seconds]
2 normalize3 -O2 [Benchmark (1000 iterations): 0.563578 seconds]

```

Wynik profilowania

1	49.18%	a.out	a.out	[.] normalize_text(char const*, long)
2	28.42%	a.out	libc.so.6	[.] __strcmp_evex
3	6.24%	a.out	libc.so.6	[.] tolower
4	1.28%	a.out	libc.so.6	[.] __mcount_internal
5	0.97%	a.out	libc.so.6	[.] _mcount
6	0.44%	a.out	libc.so.6	[.] ispunct
7	0.24%	a.out	a.out	[.] isalpha@plt
8	0.20%	a.out	a.out	[.] tolower@plt

Tutaj czas działania jest o około 50% krótszy niż w przypadku wersji poprzedniej pracującej na `string` bez optymalizacji.

W przypadku optymalizacji `-O2` czas działania jest już tylko 12% krótszy, co także pokazuje, że kompilator dobrze radzi sobie z optymalizacją pracy z napisami w Cpp.

Kod stał się znacznie mniej czytelny oraz bardziej podatny na błędy, co może nie być warte takich zmian szczególnie jeśli nie jesteśmy ograniczeni platformą.

W wyniku profilowania widzimy, że występują jedyne podstawowe operacje wykorzystywane w tej wersji algorytmu.

Wersja równoległa

W wersji równoległej użyto `std::thread` oraz podziału tekstu na równe części w ilości wątków dostępnych w systemie. Dla każdej części uruchamiana jest bazowa wersja algorytmu. W tym przypadku nie rozważałem innych typów podziału, np. pomiędzy wyrazami ani rozwiązywania problemów z łączeniem fragmentów, ponieważ ilość operacji łączenia części będzie proporcjonalna do liczby wątków i nie wpłynie to w zauważalny sposób na czas działania.

```

1 std::string normalize_text_threaded(const std::string& text) {
2     unsigned int num_threads = std::thread::hardware_concurrency();
3
4     // For very small texts or single thread systems, use the original function
5     if (text.size() < 1000 || num_threads <= 1) {
6         return normalize_text(text);
7     }

```

```

8
9 // Split the text into chunks
10 std::vector<std::string> chunks;
11 std::vector<std::thread> threads;
12 std::vector<std::string> results(num_threads);
13
14 size_t chunk_size = text.size() / num_threads;
15 for (unsigned int i = 0; i < num_threads; ++i) {
16     size_t start = i * chunk_size;
17     size_t end = (i == num_threads - 1) ? text.size() : (i + 1) * chunk_size;
18     chunks.push_back(text.substr(start, end - start));
19 }
20
21 // Process each chunk in a separate thread
22 for (unsigned int i = 0; i < num_threads; ++i) {
23     threads.emplace_back(
24         [&chunks, &results, i]() { results[i] = normalize_text(chunks[i]); });
25 }
26
27 // Wait for all threads to complete
28 for (auto& thread : threads) {
29     if (thread.joinable()) {
30         thread.join();
31     }
32 }
33
34 std::string combined_result;
35 for (const auto& result : results) {
36     combined_result += result;
37 }
38 return combined_result;
39 }

```

Czas działania dla plików o rozmiarze 100KB przy 24 wątkach

```

1 normalize4 -00 [Benchmark (1000 iterations): 1.43878 seconds]
2 normalize4 -02 [Benchmark (1000 iterations): 1.25015 seconds] > normalize1 -02

```

Czas działania dla plików o rozmiarze 100MB przy 24 wątkach

```

1 normalize4 -00 [100MB] [Benchmark (1 iterations): 0.273202 seconds]
2 normalize4 -02 [100MB] [Benchmark (1 iterations): 0.196783 seconds]

```

Wynik profilowania

1	98.43%	a.out	libc.so.6	[.] __mcount_internal
2	0.79%	a.out	libc.so.6	[.] _mcount
3	0.18%	a.out	a.out	[.] normalize_text(std::__cxx11::basic_string<char, ...
4	0.14%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
5	0.11%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
6	0.03%	a.out	libc.so.6	[.] tolower
7	0.03%	a.out	libc.so.6	[.] malloc
8	0.03%	a.out	libc.so.6	[.] __memmove_avx512_unaligned_erms
9	0.02%	a.out	a.out	[.] __gnu_cxx::__normal_iterator<char const*, ...
10	0.02%	a.out	a.out	[.] __gnu_cxx::__normal_iterator<char const*, ...
11	0.02%	a.out	a.out	[.] bool __gnu_cxx::operator!=<char const*, ...
12	0.01%	a.out	libc.so.6	[.] _int_malloc

13	0.01%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
14	0.01%	a.out	libc.so.6	[.] isalpha
15	0.01%	a.out	libstdc++.so.6.0.33	[.] std::__cxx11::basic_string<char, ...
16	0.01%	a.out	libc.so.6	[.] _int_free

Tutaj najważniejszą obserwacją jest to, że czas działania w przypadku pracy na plikach o rozmiarze 100KB i 1000 iteracjach jest gorszy niż w przypadku wersji sekwencyjnej z powodu kosztu tworzenia nowych wątków. W wersji równoległej występują także problemy z profilowaniem. W przypadku plików o rozmiarze 100MB i 1 iteracji czas działania jest lepszy, ale mimo wykorzystania 24 wątków, program był tylko 3.6 razy szybszy.

Także warto zauważyć, że w przypadku kiedy chcielibyśmy rzeczywiście rozdzielić przetwarzanie podział na fragmenty oraz ich łączenie może nie być trywialne w zależności od algorytmu.

Wnioski

Operacje na ciągach tekstowych są trudne do optymalizacji. Nie udało mi się znaleźć miejsca, w którym można by było wykorzystać wektoryzację. Powodem tego może być dosyć skomplikowana logika algorytmu zależna od stanu.

Być może rozdzielenie przetwarzania tekstu na kilka przebiegów umożliwiłoby bardziej efektywne optymalizacje, ale kosztem dodatkowych iteracji po całości tekstu.

Wersja równoległa przyniosła poprawę w przypadku dużych plików (100MB) gdzie koszt, tworzenia nowych wątków był proporcjonalnie mały do czasu przetwarzania. Przyspieszenie wynosiło około 15% na każdy dodatkowy wątek (do 24, czyli łącznie 3.6 razy szybciej), co nie jest idealne.

Najważniejszą obserwacją jest to, jak dobrze kompilator radzi sobie z optymalizacją kodu i implementacją abstrakcji w Cpp. W porównaniu z nieczytelną wersją w stylu C, wersja podstawowa przy użyciu optymalizacji -O2 była tylko 14% wolniejsza.